

JavaScript Interview Q&A

Fundamentals to Advanced — Complete Preparation Guide

Variables · Operators · Control Flow · Arrays · Objects · DOM · Functions · Closures · Async JS · ES6+

Interview-Ready · Point-wise Answers

Table of Contents

1. Variables, Data Types & Operators
2. Truthy/Falsy & Type Coercion (Implicit vs Explicit)
3. Statements vs Expressions
4. Hoisting & Scopes
5. Selection Statements: if-else, Ternary, Switch
6. Loops: for, while, do-while, for-in, for-of
7. Arrays: Creation, Access, Update, Iteration
8. Array Methods: Normal & Higher-Order
9. Objects & Array of Objects
10. DOM: Selectors, Properties, Events, Event Listeners
11. Functions: FD, FE, AF, ARRF, IIFE (5 Types) & Parameters vs Arguments
12. Callbacks, Closures & Lexical Scope
13. ES6+ Features
14. this Keyword, call/apply/bind, Prototypes & Classes (Extra Must-Know)
15. Synchronous vs Asynchronous JS, Event Loop
16. Promises & Async/Await
17. Extra Topics Often Asked (Bonus)

1. Variables, Data Types & Operators

Q1. What is a variable in JavaScript?

- A named container that stores a value in memory
- Declared using `var`, `let`, or `const`
- Value can be changed (except `const`)

Q2. Difference between var, let and const?

Feature	var	let	const
Scope	Function scope	Block scope	Block scope
Re-declare	Allowed	Not allowed	Not allowed
Re-assign	Allowed	Allowed	Not allowed
Hoisting	Hoisted, initialized as undefined	Hoisted but in TDZ	Hoisted but in TDZ

Q3. What are the data types in JavaScript?

- **Primitive:** String, Number, Boolean, Undefined, Null, Symbol, BigInt
- **Non-Primitive (Reference):** Object, Array, Function
- Primitives are stored by value; Non-primitives stored by reference

Q4. What is the difference between undefined and null?

- `undefined` — variable declared but not assigned a value (set by JS engine)
- `null` — intentional absence of value (set by developer)
- `typeof undefined` → "undefined"; `typeof null` → "object" (known JS bug)

Q5. What are operators in JavaScript? List the types.

- Arithmetic: `+` `-` `*` `/` `%` `**`
- Assignment: `=` `+=` `-=` `*=` `/=`
- Comparison: `==` `===` `!=` `!==` `>` `<` `>=` `<=`
- Logical: `&&` `||` `!`
- Bitwise: `&` `|` `^` `~` `<<` `>>`
- Ternary: `condition ? a : b`
- Nullish Coalescing: `??`
- Optional Chaining: `?.`
- `typeof`, `instanceof`, spread/rest `...`

Q6. Difference between == and ===?

- `==` — loose equality, converts types before comparing (coercion)
- `===` — strict equality, checks value AND type, no conversion
- Best practice: always use `===` to avoid bugs

Q7. What is NaN? How do you check for it?

- "Not-a-Number" — result of an invalid numeric operation
- `typeof NaN` → "number"
- `NaN === NaN` is `false` (only value not equal to itself)
- Check using `isNaN(x)` or better `Number.isNaN(x)`

2. Truthy/Falsy & Type Coercion

Q8. What are falsy values in JavaScript?

- `false`, `0`, `-0`, `""` (empty string), `null`, `undefined`, `NaN`
- Everything else is **truthy**, including `"0"`, `[]`, `{}`

Q9. What is Implicit vs Explicit type conversion?

- **Implicit (coercion)**: JS automatically converts type — e.g. `"5" + 1 // "51"`, `"5" - 1 // 4`
- **Explicit (casting)**: Developer manually converts — e.g. `Number("5")`, `String(5)`, `Boolean(1)`

Q10. Why does `[] + []` give `""` and `[] + {}` give `"[object Object]"`?

- The `+` operator triggers implicit conversion to primitive via `toString()`
- `[]`.`toString()` → `""` so `[]+[]` → `""`
- `{}`.`toString()` → `"[object Object]"`

3. Statements vs Expressions

Q11. Difference between a statement and an expression?

- **Expression:** Produces a value — e.g. `5 + 3`, `x = 10`, function call
- **Statement:** Performs an action, doesn't necessarily return a value — e.g. `if`, `for`, `function declaration`
- Expressions can be part of statements; statements cannot be part of expressions

4. Hoisting & Scopes

Q12. What is hoisting?

- JS moves variable/function **declarations** to the top of their scope during compile phase
- `var` is hoisted and initialized as `undefined`
- `let` / `const` are hoisted but stay in **Temporal Dead Zone (TDZ)** until initialized
- Function Declarations are fully hoisted (can be called before defined)
- Function Expressions/Arrow Functions are NOT hoisted with their value

Q13. What is Temporal Dead Zone (TDZ)?

- The time between entering scope and the actual declaration line
- Accessing `let` / `const` variable in this zone throws `ReferenceError`

Q14. Types of scope in JavaScript?

- **Global Scope** — accessible everywhere
- **Function Scope** — accessible only inside function (var)
- **Block Scope** — accessible only inside {} (let, const)
- **Lexical Scope** — inner functions have access to outer function's variables based on where they are physically written

5. Selection Statements: if-else, Ternary, Switch

Q15. Syntax and use of if-else?

```
if (condition) {  
  // block A  
} else if (condition2) {  
  // block B  
} else {  
  // block C  
}
```

- Executes a block based on a boolean condition

Q16. What is the ternary operator? When to use it?

- Shorthand for if-else: `condition ? valueIfTrue : valueIfFalse`
- Best for simple, single-line conditional assignments
- Avoid nesting too many ternaries — hurts readability

Q17. When to use switch over if-else?

- When comparing one variable against many fixed values
- Uses strict comparison (`===`)
- Needs `break` to prevent fall-through; `default` for no-match case

6. Loops

Q18. Types of loops in JavaScript?

- `for` — fixed number of iterations
- `while` — runs while condition is true (checked before)
- `do-while` — runs at least once (checked after)
- `for...in` — iterates over object keys / array indices
- `for...of` — iterates over iterable values (array, string, Map, Set)

Q19. Difference between `for...in` and `for...of`?

<code>for...in</code>	<code>for...of</code>
Iterates over keys/indices	Iterates over values
Used for objects (also arrays)	Used for iterables (arrays, strings, Map, Set)
Not recommended for arrays	Preferred for arrays

Q20. Difference between `break` and `continue`?

- `break` — exits the loop entirely
- `continue` — skips current iteration, moves to next

7. Arrays: Creation, Access, Update, Iteration

Q21. How to create an array?

```
let arr1 = [1, 2, 3];           // literal (preferred)
let arr2 = new Array(1, 2, 3);  // constructor
let arr3 = Array.of(5);         // [5]
let arr4 = Array.from("abc");   // ["a", "b", "c"]
```

Q22. How to access, update and delete array elements?

- Access: `arr[0]` (zero-indexed)
- Update: `arr[0] = 100`
- Delete (leaves hole): `delete arr[0]`
- Remove properly: `arr.splice(0,1)`
- Length: `arr.length`

Q23. Ways to iterate over an array?

- `for` loop with index
- `for...of` loop (values)
- `for...in` loop (indices, not recommended)
- `forEach()` method
- `map()`, `filter()`, `reduce()` (functional iteration)
- `while` loop with manual index

Q24. How to check if a variable is an array?

- `Array.isArray(arr)` — correct way
- `typeof arr` gives "object" (not reliable for arrays)

8. Array Methods: Normal & Higher-Order

Q25. Common "normal" (mutating/basic) array methods?

- `push()` / `pop()` — add/remove from end
- `shift()` / `unshift()` — remove/add from start
- `splice()` — add/remove at any index (mutates)
- `slice()` — extract portion (does NOT mutate)
- `concat()` — merge arrays
- `reverse()` , `sort()` — mutate original array
- `join()` — array to string
- `includes()` , `indexOf()` , `find()` , `findIndex()`
- `flat()` , `flatMap()`

Q26. What is a Higher-Order Function (HOF)? Give array examples.

- A function that takes another function as argument, or returns a function
- `map()` — transforms each element, returns NEW array
- `filter()` — returns new array of elements passing a condition
- `reduce()` — reduces array to a single value (accumulator)
- `forEach()` — executes callback for each element, returns undefined
- `some()` — true if ANY element passes test
- `every()` — true if ALL elements pass test
- `sort(compareFn)` — custom sort logic via callback

Q27. Difference between `map()` and `forEach()`?

<code>map()</code>	<code>forEach()</code>
Returns a new array	Returns undefined
Used when you need transformed data	Used only for side-effects (logging, DOM update)
Chainable	Not chainable

Q28. Example of `reduce()` for business logic (e.g. total price)?

```
const cart = [{price:100},{price:200},{price:50}];
const total = cart.reduce((acc, item) => acc + item.price, 0);
// total = 350
```

9. Objects & Array of Objects

Q29. How to create, access and update an object?

```
let user = { name: "Ravi", age: 25 };
user.age;           // access - dot notation
user["name"];       // access - bracket notation
user.age = 26;       // update
user.city = "Chennai"; // add new property
delete user.city;    // remove property
```

Q30. Ways to iterate over an object?

- `for...in` loop — iterates keys
- `Object.keys(obj)` — array of keys
- `Object.values(obj)` — array of values
- `Object.entries(obj)` — array of [key, value] pairs

Q31. How to work with an array of objects (real business logic example)?

```
let employees = [
  {id:1, name:"A", salary:30000, dept:"IT"},
  {id:2, name:"B", salary:45000, dept:"HR"},
  {id:3, name:"C", salary:60000, dept:"IT"}
];

// Filter IT dept employees
let itEmp = employees.filter(e => e.dept === "IT");

// Get names only
let names = employees.map(e => e.name);

// Total salary
let totalSalary = employees.reduce((sum,e)=> sum + e.salary, 0);

// Find employee with highest salary
let topEarner = employees.reduce((max,e)=> e.salary > max.salary ? e : max);

// Sort by salary descending
let sorted = [...employees].sort((a,b)=> b.salary - a.salary);
```

Q32. Difference between shallow copy and deep copy of an object?

- **Shallow copy:** copies only first level; nested objects still shared by reference — `{...obj}`, `Object.assign({}, obj)`
- **Deep copy:** fully independent copy including nested objects — `structuredClone(obj)` or `JSON.parse(JSON.stringify(obj))`

10. DOM: Selectors, Properties, Events, Event Listeners

Q33. What is the DOM?

- Document Object Model — tree-like representation of an HTML page
- Allows JavaScript to read/modify HTML structure, style, and content dynamically

Q34. Common DOM selector methods?

- `document.getElementById("id")` — single element
- `document.getElementsByClassName("cls")` — HTMLCollection (live)
- `document.getElementsByTagName("tag")`
- `document.querySelector(".cls")` — first match (CSS selector)
- `document.querySelectorAll(".cls")` — NodeList (static) of all matches

Q35. Difference between NodeList and HTMLCollection?

- HTMLCollection — live (auto-updates with DOM changes), from `getElementsBy*`
- NodeList — mostly static, from `querySelectorAll` (except `childNodes` which is live)

Q36. Common DOM properties to manipulate elements?

- `innerHTML` — get/set HTML content
- `textContent` — get/set plain text (safer, ignores tags)
- `value` — for form inputs
- `style.property` — inline CSS, e.g. `el.style.color = "red"`
- `classList.add/remove/toggle/contains` — manage classes
- `setAttribute()` / `getAttribute()`

Q37. How to create and add elements to the DOM?

```
let li = document.createElement("li");
li.textContent = "New Item";
document.querySelector("ul").appendChild(li);
// remove: li.remove();
```

Q38. What is an event and event listener?

- Event — an action triggered by user or browser (click, submit, keydown, load...)
- Event Listener — function that "listens" for an event and reacts
- Syntax: `element.addEventListener("click", handlerFn)`
- `removeEventListener()` to detach a listener

Q39. What is event bubbling and event capturing?

- **Bubbling:** event travels from target element UP to ancestors (default)
- **Capturing:** event travels from root DOWN to target (set 3rd param `true`)
- `event.stopPropagation()` — stops further bubbling/capturing
- `event.preventDefault()` — stops default browser behavior (e.g. form submit)

Q40. What is event delegation?

- Attaching a single listener to a parent to handle events from its children
- Uses event bubbling + `event.target` to identify the actual clicked child
- Efficient for dynamically added elements and long lists

11. Functions: 5 Types, Parameters vs Arguments

Q41. What are the different ways to create a function in JS (5 types)?

```
// 1. Function Declaration (FD) - hoisted
function add(a,b) { return a+b; }

// 2. Function Expression (FE) - not hoisted
const sub = function(a,b) { return a-b; };

// 3. Arrow Function (AF)
const mul = (a,b) => a*b;

// 4. Named Function Expression
const div = function divide(a,b) { return a/b; };

// 5. IIFE (Immediately Invoked Function Expression)
(function() { console.log("runs immediately"); })();

// (Also: Function constructor - rarely used)
const f = new Function("a","b","return a+b");
```

Q42. Difference between Function Declaration and Function Expression?

- Declaration is hoisted completely (usable before definition)
- Expression is only hoisted as variable (undefined), not usable before assignment
- Declaration has a name always; Expression can be anonymous

Q43. Difference between Arrow Function and Regular Function?

Arrow Function	Regular Function
No own <code>this</code> (lexical, inherits from parent)	Has own <code>this</code> (depends on how it's called)
Cannot be used as constructor (no <code>new</code>)	Can be used as constructor
No <code>arguments</code> object	Has <code>arguments</code> object
Implicit return for single expression	Needs explicit <code>return</code>
Cannot be a generator	Can be a generator (function*)

Q44. Difference between Parameters and Arguments?

- **Parameters** — variables listed in the function definition (placeholders)
- **Arguments** — actual values passed when calling the function
- Example: `function greet(name)` → name is parameter; `greet("Ravi")` → "Ravi" is argument

Q45. What are default parameters and rest parameters?

```
function greet(name = "Guest") { return `Hi ${name}`; } // default

function sum(...nums) {                                // rest
  return nums.reduce((a,b)=>a+b, 0);
}
sum(1,2,3,4); // 10
```

12. Callbacks, Closures & Lexical Scope

Q46. What is a callback function?

- A function passed as an argument to another function, executed later
- Used in array methods, event listeners, and async operations (setTimeout, API calls)
- Can lead to "callback hell" — nested callbacks — solved with Promises/async-await

Q47. What is a closure? Give an example.

- A function that "remembers" variables from its outer (lexical) scope even after the outer function has finished executing
- Formed automatically whenever a function is created inside another function

```
function counter() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}  
const inc = counter();  
inc(); // 1  
inc(); // 2 (count is preserved via closure)
```

Q48. Practical uses of closures?

- Data privacy / encapsulation (module pattern)
- Creating function factories (e.g. multiplier generators)
- Memoization / caching results
- Maintaining state in event handlers, setTimeout loops

Q49. What is Lexical Scope?

- A variable's scope is determined by its physical location in the source code
- Inner functions can access variables of outer functions, not vice-versa
- This is the foundation that makes closures possible

13. ES6+ Features

Q50. List key ES6+ features asked in interviews.

- `let` / `const`, block scoping
- Arrow functions
- Template literals: ``Hello ${name}``
- Destructuring (array & object)
- Spread `...` and Rest `...` operators
- Default parameters
- Classes and `extends` / inheritance
- Modules — `import` / `export`
- Promises, `async/await`
- Map, Set, WeakMap, WeakSet
- Symbol, Generators (function*)
- Optional chaining `?.` and Nullish coalescing `??`

Q51. Example of destructuring?

```
const {name, age} = {name:"Ravi", age:25}; // object destructuring
const [a, b, ...rest] = [1,2,3,4,5];      // array destructuring
// a=1, b=2, rest=[3,4,5]
```

Q52. Difference between Spread and Rest operator?

- Both use `...` syntax but behave oppositely
- **Spread** — expands an array/object into individual elements — `[...arr1, ...arr2]`
- **Rest** — collects multiple elements into a single array — `function f(...args){}`

Q53. Map vs Set vs Object vs Array?

- **Map** — key-value pairs, keys can be ANY type, maintains insertion order
- **Set** — collection of unique values only
- **Object** — key-value pairs, keys are strings/symbols only
- **Array** — ordered list of values, indexed numerically

14. this, call/apply/bind, Prototypes & Classes

Q54. What does 'this' refer to in JavaScript?

- Refers to the object that is currently "executing"/calling the function
- In a regular function — depends on how it's called (global/object/new)
- In an arrow function — inherited lexically from surrounding scope
- In strict mode, standalone function call → `this` is `undefined` (not global object)

Q55. Difference between call(), apply(), and bind()?

- All three set the value of `this` explicitly
- `call(thisArg, arg1, arg2...)` — invokes function immediately, args passed individually
- `apply(thisArg, [argsArray])` — invokes immediately, args passed as array
- `bind(thisArg)` — returns a NEW function with `this` bound, does NOT invoke immediately

Q56. What is prototypal inheritance?

- Every object has an internal link (`__proto__`) to another object called its prototype
- If a property/method isn't found on the object, JS looks up the prototype chain
- Enables inheritance and method sharing without classes

Q57. ES6 Classes — how do they relate to prototypes?

- Classes are "syntactic sugar" over prototype-based inheritance
- `class Animal { speak(){} }` and `class Dog extends Animal{}` use `extends` + `super()`
- Methods defined in a class are added to the prototype, not each instance

15. Synchronous vs Asynchronous JS & Event Loop

Q58. Difference between synchronous and asynchronous code?

- **Sync:** code executes line by line, each line waits for the previous to finish (blocking)
- **Async:** allows long-running tasks (API calls, timers, file I/O) to run in background without blocking main thread
- JavaScript is single-threaded but achieves async behavior via the Event Loop

Q59. Explain the Event Loop, Call Stack, and Task Queues.

- **Call Stack:** executes functions one at a time (LIFO)
- **Web APIs:** browser handles async tasks (setTimeout, fetch, DOM events)
- **Microtask Queue:** Promises, `queueMicrotask` — HIGH priority
- **Macrotask (Callback) Queue:** setTimeout, setInterval — LOWER priority
- **Event Loop:** continuously checks if Call Stack is empty, then pushes Microtasks first, then Macrotasks

Q60. Why does setTimeout(fn, 0) not run immediately?

- It's pushed to the macrotask queue, not executed synchronously
- Only runs after the current call stack is empty AND all microtasks are processed

16. Promises & Async/Await

Q61. What is a Promise? What states can it have?

- An object representing the eventual completion (or failure) of an async operation
- States: **Pending** → **Fulfilled** (resolved) or **Rejected**
- Once settled (fulfilled/rejected), state cannot change again

```
const p = new Promise((resolve, reject) => {
  let success = true;
  success ? resolve("Done") : reject("Failed");
});
p.then(res => console.log(res)).catch(err => console.log(err));
```

Q62. Explain then(), catch(), finally().

- `.then()` — handles resolved value
- `.catch()` — handles rejected/error value
- `.finally()` — runs regardless of outcome (cleanup)
- These are chainable, each returns a new Promise

Q63. Promise combinator methods — Promise.all / allSettled / race / any?

- `Promise.all()` — waits for ALL to resolve; rejects fast if ANY fails
- `Promise.allSettled()` — waits for all, returns status of each (no early rejection)
- `Promise.race()` — settles as soon as the FIRST promise settles (resolve or reject)
- `Promise.any()` — settles as soon as the FIRST one resolves; fails only if ALL reject

Q64. What is async/await? How does it relate to Promises?

- `async` function always returns a Promise
- `await` pauses execution inside the async function until the Promise settles
- Syntactic sugar over Promises — makes async code look synchronous, easier to read
- Errors handled using `try...catch` instead of `.catch()`

```
async function getData() {
  try {
    const res = await fetch("/api/data");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error:", err);
  }
}
```

Q65. What is Callback Hell and how do Promises/async-await solve it?

- Callback Hell — deeply nested callbacks, hard to read/maintain ("pyramid of doom")
- Promises flatten this using `.then()` chaining
- async/await removes nesting entirely, code reads top-to-bottom like sync code

17. Extra Topics Often Asked (Bonus)

Q66. What is the difference between deep and shallow equality checks?

- Shallow — compares only top-level (reference for objects/arrays)
- Deep — recursively compares all nested values (needs custom function or lodash `isEqual`)

Q67. What is strict mode ('use strict')?

- Opts into a restricted variant of JS — catches silent errors, disallows undeclared variables
- Makes `this` undefined in plain function calls instead of global object

Q68. What is debouncing and throttling?

- **Debounce:** delays function execution until after a pause in event firing (e.g. search input)
- **Throttle:** ensures a function runs at most once every fixed interval (e.g. scroll events)

Q69. What are Generators (function*)?

- Special functions that can pause and resume execution using `yield`
- Return an iterator; called with `.next()`
- Useful for lazy evaluation, custom iterators, and controlling async flow

Q70. What is JSON and its main methods?

- JSON — lightweight data-interchange text format
- `JSON.stringify(obj)` — object → JSON string
- `JSON.parse(str)` — JSON string → object

Q71. What is memory leak / garbage collection in JS?

- JS engine automatically frees memory of objects no longer reachable ("Mark and Sweep" algorithm)
- Memory leaks happen via forgotten timers, detached DOM references, unremoved event listeners, global variables

Q72. What are modules (import/export)?

- Split code into reusable files
- `export default` / `export {name}` — export from a file
- `import x from './file.js'` — import into another file
- Helps maintainability, avoids polluting global scope

Q73. Difference between synchronous error handling and async error handling?

- Sync: `try { } catch(e) { }`
- Promise-based async: `.catch(err => {})`
- Async/await: wrap with `try/catch` around the `await` call

Tip: In interviews, always explain answers with a short code snippet if time permits — it shows practical understanding, not just theory.